

SPORT DATA SOLUTION

Documentation Technique POC Avantages Sportifs

Version 1.0 | Juin 2026 | Confidentiel

Projet	POC Avantages Sportifs
Entreprise	Sport Data Solution
Version	1.0
Date	Juin 2026
Périmètre	161 salariés — Environnement Docker / Windows 11
Statut	POC — Proof of Concept

Table des matières

Table des matières	2
1. Présentation du projet	4
1.1 Contexte	4
1.2 Objectifs du POC	4
1.3 Périmètre	4
2. Architecture globale	5
2.1 Vue d'ensemble	5
2.2 Réseau Docker	5
2.3 Base de données SQL Server	5
3. Flow 01 Strava (DC1) — Collecte OAuth	6
3.1 Description	6
3.2 Conteneurs	6
3.3 Flux OAuth — Séquence	7
4. Flow 01 Slack (DC2) — Notification temps réel	8
4.1 Description	8
4.2 Conteneurs	8
4.3 Flux de notification — Séquence	8
5. Flow 01 PowerBI (DC3) — ETL et calcul des primes	9
5.1 Description	9
5.2 Conteneurs	9
5.3 Règles métier	9
5.4 Validation Google Maps	10
6. Orchestration Kestra	11
6.1 Architecture	11
6.2 Flows	11
6.3 Gestion des secrets	11
6.4 Rechargement automatique des flows	12
7. Variables d'environnement	13
7.1 flow_01_Strava/.env	13
7.2 flow_01_PowerBI/.envFolder	13
7.3 flow_01_PowerBI/.env	13
7.4 dockerCompose_Kestra/.env	13
8. Démarrage du projet	15
8.1 Prérequis	15
8.2 Ordre de démarrage	15
8.3 Remplir les fichiers .env	15
8.4 Envoi des emails d'invitation	15
9. Dépendances techniques	16

9.1 Images Docker de base	16
9.2 Pilote ODBC	16
9.3 Dépendances Python par conteneur.....	16
9.4 Stack .NET (webapp)	16
10. Points d'attention et évolutions	17
10.1 Limitations du POC.....	17
10.2 Évolutions recommandées pour la production	17
10.3 Sécurité	17

1. Présentation du projet

1.1 Contexte

Sport Data Solution développe un POC nommé "Avantages Sportifs" visant à automatiser le calcul de deux avantages employés à partir des données d'activité physique Strava :

- Prime de trajet : 5% du salaire annuel brut pour les salariés se déplaçant sportivement (marche/running, vélo/trottinette)
- 5 Jours bien-être : 5 jours de congés supplémentaires pour les salariés ayant enregistré au minimum 15 activités physiques sur les 12 derniers mois

1.2 Objectifs du POC

- Collecter les données d'activité Strava des employés via OAuth 2.0
- Nettoyer et valider les données RH et sportives
- Calculer automatiquement les primes éligibles
- Notifier les équipes via Slack lors de nouvelles activités
- Visualiser les KPIs dans PowerBI

1.3 Périmètre

Critère	Valeur
Salariés concernés	161 employés
Environnement	Docker Desktop / Windows 11
Base de données	SQL Server 2022 (conteneurisé)
Orchestration	Kestra Community Edition
Visualisation	Microsoft PowerBI

2. Architecture globale

2.1 Vue d'ensemble

Le projet est structuré en 3 flows Docker Compose indépendants, reliés par un réseau Docker partagé et orchestrés par Kestra :

Flow	Rôle	Conteneurs
flow_01_Strava (DC1)	Collecte OAuth et stockage	sqlserver, init_db, send_email, webapp, getdata
flow_01_Slack (DC2)	Notification temps réel via CDC	debezium, slack
flow_01_PowerBI (DC3)	ETL et calcul des primes	strava_export, strava_testexport, strava_calcul_primes
dockerCompose_Kestra	Orchestration des flows	kestra, postgres

2.2 Réseau Docker

Les flows communiquent via un réseau Docker externe partagé (strava_network, driver: bridge). Ce réseau est créé par flow_01_Strava et référencé comme external: true dans les autres flows. flow_01_PowerBI utilise le réseau nommé flow_01_strava_strava_network.

2.3 Base de données SQL Server

Tables principales gérées dans la base StravaDb :

Table	Description
StravaTokens	Tokens OAuth par employé (IdEmploye, AthletId, AccessToken, RefreshToken, ExpiresAt)
StravaActivities	Activités sportives (Id, IdEmploye, AthletId, Name, SportType, StartDate, SlackMessage)
Donnees_RH_Nettoye	Données RH nettoyées (ID_salarie, Nom, Prenom, BU, Salaire_brut, Moyen_deplacement)
Donnees_Sportive_Nettoye	Sport déclaré par salarié
Prime_Trajet	Résultats du calcul de la prime de trajet
Prime_BienEtre	Résultats du calcul des 5 jours bien-être
StravaActivities_TestData	Activités de test générées pour les 161 salariés

Le CDC (Change Data Capture) est activé sur StravaTokens et StravaActivities pour alimenter Debezium.

3. Flow 01 Strava (DC1) — Collecte OAuth

3.1 Description

Ce flow gère l'ensemble du processus d'autorisation OAuth Strava et de collecte des activités sportives des salariés.

3.2 Conteneurs

Conteneur 1 — sqlserver

- Image : mcr.microsoft.com/mssql/server:2022-latest
- Rôle : Base de données SQL Server avec CDC activé
- Port exposé : 1433
- Volumes : données persistées dans le volume sqldata

Conteneur 2 — init_db (Python 3.11-slim)

- Rôle : Initialisation one-shot de la base de données
- Script : create_table_cdc.py
 - Crée la base StravaDb si elle n'existe pas
 - Crée les tables StravaTokens et StravaActivities avec la colonne IdEmploye
 - Active le CDC sur les deux tables via sp_cdc_enable_table
 - **Crée les utilisateurs SQL Server** : powerbi_user (rôle db_datareader, accès lecture seule aux rapports)
 - **Crée les utilisateurs SQL Server** : debezium_user (rôle db_datareader + VIEW DATABASE STATE, port 1433)
- Exécution : one-shot (restart: "no")

Conteneur 3 — send_email (Python 3.11-slim)

- Rôle : Envoi des emails d'invitation OAuth aux salariés
- Script : 1.00_SendEmail.py
 - Lit la liste des destinataires depuis destinataires.xlsx (colonnes : Nom, Prenom, Email, Id)
 - Génère un lien OAuth Strava personnalisé avec le paramètre state=IdEmploye
 - Envoie un email HTML via SMTP GMX
- Variables d'environnement : GMX_USER, GMX_PASSWORD, STRAVA_CLIENT_ID, STRAVA_CALLBACK_URL

Conteneur 4 — webapp (ASP.NET Core MVC)

- Rôle : Serveur OAuth — reçoit le callback Strava
- Port exposé : 44350
- Composants principaux :
 - HomeController.cs : gère le callback OAuth, extrait state (IdEmploye) et code
 - StravaTokenRepository.cs : sauvegarde les tokens via MERGE SQL avec IdEmploye et AthletelId
 - StravaToken.cs : modèle de données
 - DebeziumController.cs : endpoint HTTP pour recevoir les événements Debezium

Conteneur 5 — getdata (Python 3.11-slim)

- Rôle : Synchronisation périodique des activités Strava
- Script : 1.00_getData.py
 - Lit tous les athlètes depuis StravaTokens (avec IdEmploye)
 - Rafraîchit les tokens expirés automatiquement via l'API Strava
 - Appelle l'API Strava v3 (/athlete/activities) avec pagination

- Insère les activités via MERGE dans StravaActivities (avec IdEmploye)

3.3 Flux OAuth — Séquence

Etape	Action
1	Le salarié reçoit un email avec son lien personnalisé contenant &state=IdEmploye
2	Il autorise l'application sur strava.com
3	Strava redirige vers /Home/Callback?code=AUTH_CODE&state=IdEmploye
4	La webapp échange le code contre les tokens et les sauvegarde avec IdEmploye
5	Le conteneur getdata synchronise les activités toutes les 2 minutes via Kestra

4. Flow 01 Slack (DC2) — Notification temps réel

4.1 Description

Ce flow surveille les nouvelles activités en base de données via Change Data Capture (CDC) et envoie des notifications Slack en temps réel, sans recours à Kafka.

4.2 Conteneurs

Conteneur 6 — debezium (Debezium Server)

- Rôle : Capture des changements SQL Server via CDC
- Configuration : application.properties
 - Source : SQL Server — tables dbo.StravaActivities et dbo.StravaTokens
 - Sink : HTTP vers le service Flask (http://slack:5000/events)
 - Stockage des offsets : fichier local offsets.dat (sans Kafka)
 - Stockage du schéma : fichier local schema-history.dat
 - Implémentations : FileOffsetBackingStore et FileSchemaHistory
 - **Sécurité** : Les mots de passe Debezium ne sont pas dans application.properties — ils sont injectés via les variables d'environnement DEBEZIUM_SOURCE_DATABASE_USER et DEBEZIUM_SOURCE_DATABASE_PASSWORD dans docker-compose
 - **Mode snapshot** : snapshot.mode=no_data — capture le schéma au démarrage et diffuse uniquement les changements à partir du moment du démarrage (aucune donnée historique capturée)
 - **⚠ Reset LSN stale** : En cas de recreation du conteneur SQL Server, supprimer offsets.dat et schema-history.dat pour forcer un nouveau snapshot (sinon Debezium attend un LSN inexistant en silence)

Conteneur 7 — slack (Python 3.11-slim + Flask)

- Rôle : Serveur HTTP intermédiaire entre Debezium et Slack
- Script : 1.00_startSlack.py
 - Serveur Flask sur le port 5000
 - Endpoint POST /events : reçoit les événements CDC de Debezium
 - Filtre uniquement les insertions (op="c") sur la table StravaActivities
 - Envoie un message Slack via Webhook (athlète, activité, type, date)
 - Met à jour SlackMessage=1 dans StravaActivities après envoi

4.3 Flux de notification — Séquence

Etape	Action
1	Une nouvelle activité est insérée dans StravaActivities par le conteneur getdata
2	SQL Server CDC détecte le changement et l'écrit dans les tables CDC système
3	Debezium Server capture l'événement et l'envoie en HTTP au service Slack
4	Flask reçoit le payload JSON et extrait les données de l'activité
5	Un message est posté sur le canal Slack via Webhook
6	SlackMessage est mis à jour à 1 dans StravaActivities (évite les doublons)

5. Flow 01 PowerBI (DC3) — ETL et calcul des primes

5.1 Description

Ce flow extrait, nettoie, valide et transforme les données pour alimenter PowerBI et calculer les avantages salariaux (prime de trajet et 5 jours bien-être).

5.2 Conteneurs

Conteneur 1 — strava_export (Python 3.11-slim + DuckDB)

- Nom du conteneur : strava_export
- Scripts exécutés :
 - 1.00_ExportsDonnees.py : Charge les tables SQL Server dans DuckDB, ajoute rowHash et idExport, réinsère dans SQL Server
 - 2.00_NettoyageDonnees.py : Nettoyage des 5 tables (StravaTokens, StravaActivities, Donnees_RH, Donnees_Sportive, Declarations)
- Bibliothèque DuckDB utilisée en mode in-process (analytique, sans serveur dédié)
- Volume partagé /data pour les exports Excel

Conteneur 2 — strava_testexport (Python 3.11-slim)

- Nom du conteneur : strava_testexport
- Script exécuté : 1.00_TestExport.py
- Fonctions de test :
 - test_structure() : vérifie la présence des colonnes obligatoires
 - test_qualite() : vérifie l'absence de doublons et valeurs nulles
 - test_metier() : vérifie la cohérence métier des données
- Envoie un email HTML de rapport de qualité via SMTP GMX

Conteneur 3 — strava_calcul_primes (Python 3.11-slim)

- Nom du conteneur : strava_calcul_primes
- Scripts exécutés dans l'ordre :
 - 1.00_GenerateTestData.py : Génère des activités de test pour les 161 salariés (15-25 activités/an) dans StravaActivities_TestData
 - 2.00_ValidateDeclarations.py : Valide les déclarations de mode de déplacement via l'API Google Maps Distance Matrix
 - 3.00_PrimeTrajet.py : Calcule la prime de trajet, exporte vers Prime_Trajet et /data/Prime_Trajet_YYYYMMDD.xlsx
 - 4.00_PrimeBienEtre.py : Calcule les 5 jours bien-être, exporte vers Prime_BienEtre et /data/Prime_BienEtre_YYYYMMDD.xlsx

5.3 Règles métier

Avantage	Règle de calcul
Prime de trajet	5% du salaire brut annuel pour les salariés déclarant marche/running ou vélo/trottinette comme moyen de déplacement
5 Jours bien-être	5 jours de congés supplémentaires pour les salariés avec >= 15 activités sur les 12 derniers mois glissants

5.4 Validation Google Maps

Le script 2.00_ValidateDeclarations.py appelle l'API Distance Matrix pour vérifier la cohérence des déclarations de mode de transport :

Mode déclaré	Mode API	Distance max
Marche / Running	walking	15 km
Vélo / Trottinette	bicycling	25 km
Transports / Véhicule	driving / transit	Aucune limite

6. Orchestration Kestra

6.1 Architecture

Kestra Community Edition est orchestré via docker-compose avec PostgreSQL comme backend de persistance. L'interface web est accessible à l'adresse <http://localhost:8080>.

Paramètre	Valeur
URL interface	http://localhost:8080
Namespace	avantages_sportifs
Backend	PostgreSQL 15
Authentification	Désactivée (basic-auth: false)

6.2 Flows

flow_getdata.yaml

- Déclencheur : toutes les 2 minutes (cron: */2 * * * *)
- Concurrence : CANCEL — une seule exécution simultanée autorisée
- Retry : 3 tentatives avec 15 secondes d'intervalle
- Séquence : start getdata → exec 1.00_getData.py → stop getdata
- Notifications : email [OK] dans le bloc tasks: (dernière tâche onSuccess) et email [ECHEC] dans le bloc errors: (tâche onFailure) — Note : afterExecution ne résout pas les secrets Kestra en v1.3.3, d'où ce pattern tasks/errors

flow_powerbi.yaml

- Déclencheur : tous les lundis à 6h00 (cron: 0 6 * * 1)
- Retry : 3 tentatives avec 30 secondes d'intervalle
- Séquence :
 - strava_export : 3 scripts (ExportsDonnees, NettoyageDonnees, TestExport)
 - strava_testexport : 1 script (TestExport)
 - strava_calcul_primes : 4 scripts (GenerateTestData, ValidateDeclarations, PrimeTrajet, PrimeBienEtre)
- Notifications : email [OK] en fin de flow, email [ECHEC] en cas d'erreur

6.3 Gestion des secrets

En Kestra Community Edition (v1.3.3), les secrets sont gérés via des variables d'environnement préfixées SECRET_ dans le fichier dockerCompose_Kestra/.env. **IMPORTANT** : les valeurs doivent être encodées en Base64, sinon Kestra lève l'erreur "Could not decode secret, make sure it is Base64-encoded".

```
SECRET_GMX_PASSWORD=<mot_de_passe_smtp>
SECRET_GMX_USER=<email_smtp>
```

Exemple : SECRET_GMX_PASSWORD=<valeur_smtp_encodée_en_Base64>

Encodage requis : Encoder en Base64 sous PowerShell :

```
[Convert]::ToBase64String([System.Text.Encoding]::UTF8.GetBytes("valeur"))
```

Ces secrets sont accessibles dans les flows YAML via la syntaxe : {{ secret('GMX_PASSWORD') }}

6.4 Rechargement automatique des flows

Les fichiers YAML du dossier `dockerCompose_Kestra/flows/` sont montés dans le conteneur Kestra via un volume Docker. La configuration `local-files` active la surveillance automatique :

```
flows:
  local-files:
    enabled: true
    watch-directory: /app/flows
```

Toute modification d'un fichier YAML en local est automatiquement détectée et appliquée sans redémarrage du conteneur.

7. Variables d'environnement

Tous les fichiers .env et .envFolder sont exclus du dépôt git via .gitignore. Des fichiers .env.example doivent être fournis.

7.1 flow_01_Strava/.env

Variable	Usage
GMX_USER / GMX_PASSWORD	Credentials SMTP pour l'envoi des emails d'invitation
EMAIL_TO	Adresse email de réception des rapports
STRAVA_CLIENT_ID	Identifiant de l'application Strava Developer
STRAVA_CLIENT_SECRET	Secret de l'application Strava Developer
STRAVA_CALLBACK_URL	URL de callback OAuth (http://localhost:44350/Home/Callback)
SQL_SERVER / SQL_DATABASE	Coordonnées de la base SQL Server
SQL_USER / SQL_PASSWORD	Credentials SQL Server

7.2 flow_01_PowerBI/.envFolder

Variable	Usage
SQL_SERVER / SQL_DATABASE	Coordonnées SQL Server partagées par les 3 conteneurs PowerBI
SQL_USER / SQL_PASSWORD	Credentials SQL Server
PATHDATA	Chemin du dossier de données partagé (/data/)

7.3 flow_01_PowerBI/.env

Variable	Usage
EMAIL_USER / EMAIL_PASSWORD	Credentials SMTP pour les rapports de tests
EMAIL_SMTP_SERVER / PORT	Serveur SMTP (ex: mail.gmx.com:587)
GOOGLE_MAPS_API_KEY	Clé API Google Maps Distance Matrix pour la validation des déclarations

7.4 dockerCompose_Kestra/.env

Variable	Usage
SECRET_GMX_PASSWORD	Mot de passe SMTP — injecté comme secret Kestra

SECRET_GMX_USER

Email SMTP — injecté comme secret Kestra

8. Démarrage du projet

8.1 Prérequis

- Docker Desktop installé et démarré
- Docker TCP activé sur le port 2375 (requis pour Kestra sur Windows)
- Compte Strava Developer avec Client ID et Secret
- Compte Google Cloud avec l'API Distance Matrix activée
- Webhook Slack configuré sur le canal cible
- Node.js (optionnel, pour régénérer la documentation)

8.2 Ordre de démarrage

Ordre	Commande	Description
1	cd flow_01_Strava && docker-compose up -d	Démarre SQL Server, init DB, webapp OAuth et getdata
2	cd flow_01_Slack && docker-compose up -d	Démarre Debezium et le service Flask Slack
3	cd flow_01_PowerBI && docker-compose up -d	Démarre les 3 conteneurs ETL (restent en attente)
4	cd dockerCompose_Kestra && docker-compose up -d	Démarre Kestra et PostgreSQL — active les flows YAML

8.3 Remplir les fichiers .env

- Copier les fichiers .env.example en .env dans chaque dossier
- Renseigner les credentials SQL Server (à remplacer en production)
- Renseigner les credentials SMTP GMX
- Renseigner le Client ID et Secret Strava
- Renseigner la clé API Google Maps
- Renseigner l'URL du Webhook Slack

8.4 Envoi des emails d'invitation

Après démarrage de flow_01_Strava, lancer manuellement le conteneur send_email pour envoyer les emails d'invitation OAuth aux 161 salariés :

```
docker exec strava_send_email python /scripts/1.00_SendEmail.py
```

9. Dépendances techniques

9.1 Images Docker de base

Image	Usage
python:3.11-slim	Tous les conteneurs Python (Debian Bookworm)
mcr.microsoft.com/mssql/server:2022	Base de données SQL Server
kestra/kestra:latest	Orchestrateur de workflows
postgres:15	Backend de persistance Kestra
debezium/server:latest	Serveur CDC Debezium

9.2 Pilote ODBC

ODBC Driver 18 for SQL Server est installé via apt dans tous les conteneurs Python. La procédure d'installation dans le Dockerfile est la suivante :

- Ajout du dépôt Microsoft packages.microsoft.com
- Import de la clé GPG via gpg --dearmor (compatibilité Debian 12/13)
- Installation des packages msodbcsql18 et unixodbc-dev

9.3 Dépendances Python par conteneur

Conteneur	Packages Python
strava_export / init_db	duckdb==1.1.3, pyodbc==5.2.0, pandas==2.2.3, openpyxl==3.1.5, sqlalchemy==2.0.36, requests==2.32.3
strava_testexport	pyodbc==5.2.0, pandas==2.2.3, sqlalchemy==2.0.36
strava_calcul_primes	pyodbc==5.2.0, pandas==2.2.3, sqlalchemy==2.0.36, openpyxl==3.1.5, requests==2.32.3
slack	flask, requests, pyodbc
getdata / send_email	requests, pyodbc, pandas, openpyxl

9.4 Stack .NET (webapp)

- Framework : ASP.NET Core MVC — .NET 10
- Microsoft.Data.SqlClient — connexion SQL Server
- System.Text.Json — parsing JSON des réponses Strava

10. Points d'attention et évolutions

10.1 Limitations du POC

- Les credentials SQL Server (utilisateur sa) sont injectés via variables d'environnement (SQL_PASSWORD dans .env) — aucun mot de passe hardcodé dans le code depuis la réécriture du script init_db. En production, remplacer le compte sa par un compte applicatif dédié
- StravaActivities_TestData utilise IdEmploye comme Athleteld (pas de vrais tokens Strava pour les données de test)
- Le calcul des primes avec if_exists="replace" écrase les résultats précédents (pas d'historique des calculs)
- Kestra Community Edition ne dispose pas d'interface de gestion des secrets native
- TrustServerCertificate=yes dans les connexions ODBC : à remplacer par un vrai certificat en production

10.2 Évolutions recommandées pour la production

Domaine	Evolution
Sécurité SQL	Remplacer le compte SA par un compte SQL dédié avec droits limités (GRANT SELECT, INSERT uniquement)
Historisation	Passer if_exists="replace" à if_exists="append" avec filtre Annee_calcul pour historiser les primes
Mapping Strava	Ajouter une table de mapping Athleteld <-> IdEmploye pour les vraies données Strava en production
Secrets Kestra	Migrer vers Kestra Enterprise pour la gestion native et sécurisée des secrets
Monitoring tokens	Ajouter un monitoring des tokens révoqués avec alerte si le rafraîchissement devient impossible
Flow Strava	Créer un flow Kestra pour flow_01_Strava (orchestrer l'envoi emails + la collecte getdata)

10.3 Sécurité

- Tous les fichiers .env et .envFolder sont exclus du dépôt git via .gitignore
- Les credentials sont chargés exclusivement via variables d'environnement (os.getenv) — aucun hardcoding
- Le paramètre state OAuth est validé avec int.TryParse côté C# pour prévenir les injections
- Les connexions SQL utilisent des requêtes paramétrées (pas de concaténation de chaînes)
- TrustServerCertificate=yes à remplacer par un vrai certificat TLS en production
- Les mots de passe Debezium sont injectés via DEBEZIUM_SOURCE_DATABASE_USER et DEBEZIUM_SOURCE_DATABASE_PASSWORD dans docker-compose — non présents dans application.properties
- Les utilisateurs SQL Server créés par init_db ont des droits minimaux : powerbi_user (db_datareader uniquement) et debezium_user (db_datareader + VIEW DATABASE STATE uniquement)